

PERSISTENCIA_DATOS

Persistencia de datos — Backend S.I.G.I

Integrantes: Hawk Durant, Emilio Jaramillo, Rodrigo Candia

1. Enfoque general

Implementamos persistencia con **JPA (Java Persistence API)** sobre **MySQL 8**. No desarrollamos **procedimientos almacenados (SP)** ni funciones en la base: toda la lógica de acceso está en repositorios Spring Data y en las entidades `@Entity` de cada microservicio.

Esto nos pareció adecuado para un proyecto Spring Boot porque:

- Los repositorios se generan solos a partir del nombre del método (`findByEmail` , `findByEstadoOrderBy...`).
 - Hibernate puede crear o actualizar tablas en desarrollo con `spring.jpa.hibernate.ddl-auto=update` .
 - Las pruebas unitarias mockean los repositorios sin necesitar MySQL levantado.
-

2. Database per service

Aunque usamos **un contenedor MySQL** en Docker, cada microservicio apunta a una **base de datos lógica distinta**:

Microservicio	Base de datos	Entidades principales
servicio-usuario	db_usuario	Usuario
servicio-reporte	db_reporte	Reporte
servicio-ubicacion	db_ubicacion	Geocache
servicio-emergencia	db_emergencia	Emergencia
servicio-recurso	db_recurso	Recurso
servicio-notificacion	db_notificacion	Notificacion
servicio-empleo	db_empleo	Empleo , Postulacion
servicio-media	db_media	MediaArchivo

El script `docker/mysql-init/01-databases.sql` crea todas las bases al iniciar el contenedor por primera vez.

3. Cómo configuramos JPA en cada servicio

En `application.yml` de cada módulo (valores típicos):

```
spring:
  datasource:
    url: jdbc:mysql://${MYSQL_HOST:localhost}:${MYSQL_PORT:3306}/db_<servicio>
    username: ${MYSQL_USER:root}
    password: ${MYSQL_PASSWORD:root}
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect
```

ddl-auto=update : Hibernate agrega columnas y tablas nuevas según las entidades. En producción real usaríamos migraciones con Flyway o Liquibase; para el caso Duoc nos bastó con update.

4. Ejemplo de entidad y repositorio

Entidad `Reporte` (servicio-report)

- Tabla `reportes` con enums `EstadoReporte` y `PrioridadReporte` mapeados como `STRING`.
- Campos de auditoría: `fechaReporte`, `fechaValidacion`, `operadorId`.
- Relación lógica con usuario vía `usuarioId` (Long), sin FK cross-database.

Repositorio

```
public interface ReporteRepository extends JpaRepository<Reporte, Long> {
    List<Reporte>
        findByEstadoOrderByPrioridadDescFechaReporteAsc(EstadoReporte estado);
    long countByDescripcionStartingWithAndFechaReporteAfter(String prefijo,
        LocalDateTime desde);
}
```

Spring Data implementa el SQL a partir del nombre del método; no escribimos JDBC manual.

5. Garantías de persistencia

Aspecto	Cómo lo garantizamos
Transacciones	<code>@Transactional</code> en métodos de servicio que guardan varias operaciones (validar reporte, postular a empleo, asignar recursos).
Integridad referencial local	Claves primarias autoincrementales; unicidad de email/RUT en <code>Usuario</code> .
Consistencia eventual entre servicios	Si Feign falla después de un <code>save</code> , el dato local ya quedó persistido (ej.: reporte sin coordenadas, emergencia sin notificación). Lo documentamos como trade-off MVP.
Caché de geocodificación	<code>Geocache</code> en servicio-ubicacion evita repetir llamadas costosas a OpenCage.
Archivos binarios	servicio-media guarda bytes en disco (<code>sigi.media.upload-dir</code>) y metadatos en <code>MediaArchivo</code> vía JPA.
Migración de esquema manual	<code>RollColumnMigration</code> convierte columna <code>rol</code> de ENUM a VARCHAR en servicio-usuario cuando MySQL no amplía enums sola.

6. Datos iniciales (seed)

Al arrancar en vacío, algunos servicios cargan datos demo con `CommandLineRunner`:

- **UsuariosPruebaInitializer** — Hawk, Emilio, Rodrigo y personal de emergencia.
- **DatosEjemploValleDelSol** — camiones y brigadas de ejemplo.
- **DatosEjemploEmpleos** — avisos laborales de prueba.

Solo insertan si el repositorio está vacío (`count() == 0` o `existsByEmail()`).

7. Conexión desde el host (DBeaver)

- Host: `localhost`
- Puerto: **13306** (mapeo Docker; evita conflicto con MySQL local en 3306)
- Usuario / clave: `root` / `root` (por defecto en compose)

8. Relación con las pruebas unitarias

En las pruebas **no** levantamos MySQL: mockeamos `UsuarioRepository`, `ReporteRepository`, etc. La capa JPA se valida indirectamente mediante tests de servicio que

verifican que se llamó a `save()`, `findById()` o `deleteById()` con los argumentos esperados.

Para una segunda etapa consideramos `@DataJpaTest` con H2 en memoria, pero no era requisito del sprint actual.

Documento del equipo Hawk Durant, Emilio Jaramillo y Rodrigo Candia.